News App: Beginner's Step-by-Step Code Guide

A step-by-step guide to the React news app, JWT authentication, Express API, and MongoDB Atlas.

Beginner edition

Project code explained, setup walkthrough, account flow, and troubleshooting

This guide explains the news website, how its React interface works, and how account authentication connects to the Express API and MongoDB Atlas. It is written for someone learning the project for the first time.

> **Protect your database:** Never put a real MongoDB URI, database password, or JWT secret in this guide, source code, screenshots, or a public repository. This project uses `.env` for private settings and ignores it in Git. A database password was shared during setup; rotate it in MongoDB Atlas before using the app for real accounts.

1. What the application does

The project has two programs that run together:

1. **Frontend:** React renders the news home page, navigation, sign-in/registration dialog, and footer in the browser.
2. **Backend:** Express receives the account requests, validates them, reads or writes users in MongoDB, and sets or clears a signed JWT cookie.

The browser does not connect directly to MongoDB. The browser sends HTTP requests to the backend; only the backend has the MongoDB connection string.

Request path at a glance

```
Browser (React on Vite :5173)
       |
       | /api/auth/register, /login, /me, /logout
       v
Vite development proxy
       |
       v
Express API (:5000) ---- Mongoose ---- MongoDB Atlas (userdb.users)
       |
       +---- signed JWT in an HTTP-only cookie
```

In development, Vite forwards requests beginning with `/api` to the Express server. The `dev:all` command starts both programs. MongoDB must be reachable and the values in `.env` must be valid.

2. Important folders and files

```
project-root/
|-- .env                Private local configuration; never commit this
|-- .env.example        Safe template showing which settings are required
|-- .gitignore          Excludes .env, dependencies, and build output from Git
|-- index.html          HTML shell into which React is mounted
|-- package.json        Dependencies and npm commands
|-- package-lock.json   Exact dependency versions installed by npm
|-- vite.config.js      Vite setup and the /api development proxy
|-- server/
|   |-- index.js        Starts Express and connects to MongoDB
|   |-- models/User.js  MongoDB/Mongoose user schema
|   |-- middleware/
|   |   |-- requireAuth.js  Verifies the JWT cookie on protected routes
|   |-- routes/
|       |-- auth.js     Registration, login, current-user, and logout API
|-- src/
|   |-- main.jsx        Browser entry point; mounts the React app
|   |-- App.jsx         Page layout: header, main headline, footer
|   |-- App.css         Older Vite starter styles (not used by App.jsx)
|   |-- index.css       Global colors, fonts, and page layout
|   |-- components/
|       |-- Header.jsx  Navigation, account dialog, and auth requests
|       |-- Header.css  Header, responsive menu, and account dialog styles
|       |-- Footer.jsx  Footer content and link groups
|       |-- Footer.css  Footer layout and responsive styles
|   |-- assets/         Images and starter SVG logos
|-- public/             Static icons served directly by Vite
```

`node_modules/` contains installed libraries and should not be edited by hand. `dist/` is generated when the frontend is built. Neither is application source.

3. Step one: install and configure the project

Requirements

- Node.js and npm installed on Windows.
- A MongoDB Atlas cluster.
- A MongoDB **database user** with permission to access the app database. This is not the same as your Atlas website login.
- Your current IP address allowed in Atlas **Network Access**.

Configure `.env`

1. Open a terminal in the project folder.
2. Copy the template:

```
Copy-Item .env.example .env
```

3. Open `.env` in VS Code. Fill in the Atlas **driver connection string** and make a long, random JWT secret (at least 32 characters):

```
MONGODB_URI=mongodb+srv://DB_USER:URL_ENCODED_PASSWORD@CLUSTER_HOST/userdb?retryWrites=true&w
 =majority&appName=Cluster0
JWT_SECRET=put-a-long-random-secret-here
PORT=5000
NODE_ENV=development
```

Use the exact host and connection options shown by Atlas under **Cluster0 > Connect > Drivers**. Replace the sample placeholders; do not paste angle brackets into the actual URI. The database path `userdb` selects the app database. If the password contains reserved URI characters such as `@`, `/`, `:`, `#`, or `%`, URL-encode the password portion.

4. Save `.env`. The `.gitignore` rules exclude `.env` and `.env.*` from Git, while allowing `.env.example`.
5. In Atlas, confirm the database user and Network Access settings. Never put the real URI in `src/`, `vite.config.js`, a PDF, or a public post.

Install dependencies and start both programs:

```
npm install
npm run dev:all
```

The terminal shows the API and frontend logs. Open the local URL printed by Vite (usually `http://localhost:5173`). Stop both with `Ctrl+C`.

Other useful commands:

```
npm run server  # Start only the API
npm run dev     # Start only the frontend
npm run build   # Create the production frontend in dist/
npm run lint    # Check source files for lint errors
```

4. How the frontend starts

`index.html`

This is the HTML page returned by Vite. Its `<div id="root"></div>` is an empty mount point. The module script loads `/src/main.jsx`. The browser tab title and favicon are also set here.

src/main.jsx

This file is the React entry point:

- It imports React's `StrictMode` and `createRoot`.
- It imports global `index.css`.
- It imports the main `App` component.
- `createRoot(...).render(...)` tells React to render the app inside the `root` element.

`StrictMode` adds development-time checks; it does not create a second production app.

src/App.jsx

`App` composes the page from reusable components: `<Header />`, the main headline and introductory text, and `<Footer />`. The imported `useState` and `heroImg` are currently unused leftovers from the starter template; they can be removed if the home page is later cleaned up.

src/components/Header.jsx

`Header` owns the interactive header and account dialog.

React state

- `menuOpen`: whether the mobile navigation is expanded.
- `accountOpen`: whether the registration/login dialog is visible.
- `mode`: either `register` or `login`; controls the form fields and API path.
- `form`: the values currently typed into the account form.
- `user`: the signed-in user's safe public details, or `null`.
- `message`: a validation, server, or network error shown to the user.
- `submitting`: disables the submit button while an auth request is running.

`useState` stores values that change during interaction. When a state setter runs, React renders the component again with the new state.

Restore a session on page load

The first `useEffect` calls `GET /api/auth/me` when the header mounts. The browser sends cookies with `credentials: 'include'`. If the cookie contains a valid JWT, the API returns the public user and the header shows a greeting and sign-out button. A `401` means there is no active session and the visitor remains signed out.

The `active` flag prevents updating state if the component has unmounted while the request was in progress.

Open and close the account dialog

`openAccount` clears an old error, sets the initial mode to registration, and opens the dialog. A second effect listens for Escape while the dialog is open and removes the event listener when it closes. Clicking the dark backdrop also closes the dialog.

Submit registration or login

`handleSubmit` prevents the browser's regular form submission, then sends a JSON `POST` to one of:

- `/api/auth/register` when `mode` is `register`.
- `/api/auth/login` when `mode` is `login`.

The API response includes safe user details. The actual JWT arrives as an HTTP-only cookie, so JavaScript cannot read it. On success, the header sets `user`; on failure, it displays the API error message.

HTML form attributes such as `required`, `type="email"`, and `minLength` improve the experience, but the backend repeats validation because browser checks are not a security boundary.

Sign out

`handleSignOut` sends `POST /api/auth/logout`. The server clears the cookie, then the frontend clears its user state. If the request fails, an error is displayed rather than pretending sign-out succeeded.

The navigation and Material UI icon imports at the top provide the header links and menu/search/close icons. `Header.css` styles the desktop/mobile navigation and dialog.

`src/components/Footer.jsx`

The footer uses a `footerGroups` array to describe link groups. `.map()` generates each group and link from that data, avoiding repetitive markup. Social icons come from Material UI Icons. The links currently point to page fragments such as `#privacy`; they are presentation placeholders until real pages are added.

5. How the backend starts and connects to Atlas

`server/index.js`

This is the API entry point:

1. `dotenv/config` loads local `.env` variables into `process.env`.
2. Startup stops with a clear error if `MONGODB_URI` is missing or `JWT_SECRET` is shorter than 32 characters.
3. Express is created and its `X-Powered-By` response header is disabled.
4. `express.json({ limit: '10kb' })` parses JSON request bodies and limits their size.
5. `cookieParser()` makes incoming cookies available as `request.cookies`.
6. `/api/auth` requests are sent to the authentication router.
7. The final error middleware logs unexpected server errors and returns a generic `500` message. It forwards errors if a response was already started.
8. `mongoose.connect(MONGODB_URI)` connects to Atlas before the API starts listening. If the database cannot connect, the API does not claim to be ready.
9. Express listens on `PORT`, or port `5000` when none is set.

Mongoose is the library that maps JavaScript models and queries to MongoDB collections and documents.

`server/models/User.js`

This file defines the `User` schema. Each user document contains:

- `username`: required, trimmed, 2-30 characters.
- `email`: required, trimmed, lowercased, unique, up to 254 characters.
- `passwordHash`: required and hidden from normal Mongoose queries (`select: false`).
- `phone`: required, trimmed, up to 30 characters.
- `profileType`: restricted to `reader`, `student`, `professional`, or `other`.
- `createdAt` and `updatedAt`: automatically supplied by Mongoose timestamps.

The model is named `User`; MongoDB conventionally stores these documents in the `users` collection in the `userdb` database.

6. Authentication routes, step by step

`server/routes/auth.js` defines four endpoints under `/api/auth`. `Router` lets the authentication endpoints live in their own file. `bcryptjs` hashes and checks passwords. `jsonwebtoken` signs and verifies session tokens.

Shared session helpers

The cookie options make the JWT cookie:

- `httpOnly`: JavaScript in the browser cannot read the token.
- `sameSite: 'lax'`: limits some cross-site cookie sending.
- `secure` in production: HTTPS is required in production.
- Scoped to `/api/auth`.
- Expired after seven days.

`publicUser` selects only the fields safe to send to the frontend. It deliberately omits `passwordHash`. `issueSession` signs a JWT with the user's database ID in the `sub` claim, limits it to the HS256 algorithm, gives it a seven-day expiry, and sets it as the cookie.

`POST /api/auth/register`

1. Reads `username`, `email`, `password`, `phone`, and `profileType` from the JSON body.
2. Checks types, email shape, allowed profile type, and field/password length limits.
3. Trims and lowercases email so the same address has a consistent stored form.
4. Checks for an existing email and returns `409 Conflict` if it already exists.
5. Hashes the password using bcrypt with 12 rounds. The plaintext password is never stored.
6. Creates a MongoDB user document.
7. Signs the user in immediately by setting the JWT cookie.
8. Returns `201 Created` with only the public user fields.

MongoDB's unique email index also protects against two registrations racing at the same time; duplicate-key error `11000` is translated to `409`.

`POST /api/auth/login`

1. Checks that email and password are strings and that the password is within bcrypt's byte-length limit.
2. Normalizes the email and fetches the user, explicitly selecting the normally hidden `passwordHash`.
3. `bcrypt.compare` checks the submitted password against the stored hash.
4. Wrong email or password produces the same `401` message, so the endpoint does not reveal which one was wrong.
5. Correct credentials create a new JWT cookie and return the public user.

`GET /api/auth/me`

This endpoint has `requireAuth` middleware before its handler. If the cookie is valid, it looks up the user ID from the token's `sub` claim and returns public details. If the account no longer exists, it clears the cookie and responds with `401`.

`POST /api/auth/logout`

Clears the JWT cookie using the same cookie path/security settings and returns `204 No Content`.

Error handling

Expected input and login errors have clear `400`, `401`, or `409` responses. Unexpected errors are passed to Express's error handler with `next(error)`; they are not silently treated as a successful login or registration.

7. JWT verification middleware

`server/middleware/requireAuth.js` is reusable protection for routes that should only work for signed-in users:

1. Reads `newsAuth` from the parsed cookies.
2. If no token exists, returns `401`.
3. Verifies the signature and expiry using the server-only `JWT_SECRET`.
4. On success, puts the verified claims on `request.auth` and calls `next()` so the route handler can run.
5. On an invalid or expired token, clears the stale cookie and returns `401`.

To protect a future endpoint, import this middleware and add it before the endpoint handler, for example `router.get('/saved', requireAuth, handler)`. Keep authorization checks on the server; hiding a button in React is not access control.

8. Frontend and API wiring

`vite.config.js`

Vite loads the React plugin and configures a development proxy:

```
/api/...  ->  http://localhost:5000/api/...
```

The browser uses relative URLs such as `/api/auth/login`, so it does not need to know the backend host in development. If the API port changes in `.env`, update the proxy target too.

`package.json`

`dependencies` are libraries used at runtime (React, Express, Mongoose, JWT, bcrypt, cookie parsing, and dotenv). `devDependencies` are development tools such as Vite, ESLint, and `concurrently`.

- `npm run dev`: starts Vite only.
- `npm run server`: starts the API with Node's watch mode.
- `npm run dev:all`: starts both commands together; if one exits, `concurrently --kill-others` stops the other.
- `npm run build`: builds the frontend into `dist/`.
- `npm run lint`: runs ESLint.

`package-lock.json` records exact resolved package versions. Use `npm install` after dependencies change; do not manually edit `node_modules`.

`.env.example`, `.env`, and `.gitignore`

`.env.example` contains placeholders and is safe to share. `.env` contains real credentials and is ignored by Git. Never replace the template with the real secret, remove the ignore rule, or send `.env` to someone. If a database password has been exposed, rotate it in Atlas and update `.env`.

Styling files

- `src/index.css` defines global color/font variables, page width, headings, and the optional dark color scheme.

- `src/components/Header.css` styles header actions, responsive navigation, the account modal, form fields, and error/success states.
- `src/components/Footer.css` controls footer columns, links, social icons, and the mobile layout.
- `src/App.css` contains older Vite starter selectors (counter, hero, next steps) that the current `App.jsx` does not use.

CSS custom properties such as `--text` and `--bg` are shared design values. The `@media` rules adjust the layout for narrower screens.

`eslint.config.js`

This config enables ESLint's recommended JavaScript checks, React Hooks rules, React Refresh checks, browser globals, and excludes generated `dist/`. `npm run lint` may also report old unused imports in `src/App.jsx`; those imports do not stop the Vite build.

`public/` and `src/assets/`

`public/` contains files served at fixed root URLs, such as the favicon and icon sprite. `src/assets/` contains files imported into JavaScript and processed by Vite during builds. The project includes `hero.png` and starter React/Vite SVG logos.

9. Try the account flow

1. Start the project with `npm run dev:all`.
2. Open the Vite URL from the terminal.
3. Choose **Sign in / Join** and create a test account.
4. A successful registration returns to the header as a signed-in user.
5. Refresh the browser. The header calls `/api/auth/me`; the HTTP-only cookie restores the session.
6. Sign out. The server clears the cookie and the header returns to the signed-out state.
7. Open MongoDB Atlas Data Explorer, choose `userdb`, and inspect the `users` collection. You should see the account fields and a bcrypt hash, never the plaintext password.

Do not use a real or reused password for testing.

10. API reference

Method	Path	Purpose	Success
`POST`	`/api/auth/register`	Validate and create a user; start a session	`201`
`POST`	`/api/auth/login`	Verify credentials; start a session	`200`
`GET`	`/api/auth/me`	Read current account from session cookie	`200`
`POST`	`/api/auth/logout`	Clear the session cookie	`204`

Common errors: `400` means invalid/missing input, `401` means not signed in or incorrect credentials, `409` means the email is already registered, and `500` indicates an unexpected server problem.

11. Troubleshooting

API says a setting is missing

Check that `.env` is in the project root (next to `package.json`), uses `KEY=value` syntax, and has `MONGODB_URI` and a JWT secret of at least 32 characters. Restart the API after changing `.env`.

MongoDB server selection or timeout error

Check the Atlas cluster is running, the URI host is copied correctly, the database username/password are valid, the password is URL-encoded where necessary, and your current public IP is allowed in Atlas Network Access. Confirm the database user's permissions.

Authentication requests return 404 or cannot connect

Make sure `npm run dev:all` is running and both API and Vite report ready. Ensure the Vite proxy points to the same API `PORT` from `.env`. Restart both services after changing config.

Registration says the email is already used

Email is unique and normalized to lowercase. Use another address or sign in with the existing account.

Sign-in succeeds but refresh looks signed out

Confirm requests include `credentials: 'include'`, the API and frontend are accessed through the same development site, cookies are enabled, and both processes use the same `JWT_SECRET`. Changing the JWT secret invalidates existing cookies.

Port already in use

Another program may be using the default port. Stop the other project process or choose another API port and update the `/api` proxy in `vite.config.js`.

12. Safe next steps for learning

1. Start by changing visible text in `App.jsx` or footer link data in `Footer.jsx`.
2. Change spacing or colors in component CSS and `index.css`.
3. Change one form label and follow its value from `Header.jsx` to the matching request in `auth.js`.
4. Add a new field by updating the frontend form, backend validation, Mongoose schema, and public response if it should be visible. Never include a password or hash in a public response.
5. Add protected routes with `requireAuth`, then verify both signed-in and signed-out behavior.
6. Run `npm run build` and `npm run lint` after changes. Review lint output; existing starter unused imports in `App.jsx` can be removed separately.

For production, also configure HTTPS, a production-only secret manager, stricter rate limiting, monitoring, and an appropriate cookie/CSRF policy. Do not expose the Express API directly with development settings.